

CPT304 Coursework 1 Report

Taskify Research-Led Software Enhancement

Group XX

May 2026

1 App Selection

Taskify was chosen because it is a task-management app with a simple user journey: a visitor signs up or logs in, then manages tasks on the dashboard. That made it practical to audit the project from the page templates through to the Express routes and tests. The app was also not already polished to the coursework baseline. It had issues in accessibility, backend form handling, persistence, test coverage, and compliance pages. Those gaps made it a good project for small, traceable fixes rather than a broad rewrite.

2 Authentication Form Fields Relied on Placeholder Text

2.1 Detection

The issue was first found while checking `views/signup.ejs`. The sign-up and login inputs had placeholder text, but the user-entered fields did not have their own visible labels or matching `id/for` pairs. This was easy to miss because the page looked acceptable before typing. The problem appeared once text was entered: the placeholder disappeared, leaving the filled username field without a visible field name. The browser evidence in Figure 1 shows that exact before and after state.

2.2 Literature

W3C WAI's form tutorial explains that labels should identify form controls and should be programmatically connected to the matching input, usually by pairing `label for` with an input `id` [1]. WebAIM gives the same practical warning: placeholder text may be useful as a hint, but it should not replace a real label because it disappears during entry and can make review harder [2]. This means the fix needed to be semantic, not just visual.

2.3 Implementation

The original field relied on the placeholder to describe the input:
Before fix: placeholder-only email field.

```
<input type="email" name="SignUpEmail"
  placeholder="Email" required="true" />
```

The updated page keeps the hint text, but adds a real label and a stable association:
After fix: visible label connected to the input.

```
<label class="field-label" for="signup-email">Email</label>
<input id="signup-email" type="email" name="SignUpEmail"
  autocomplete="email" placeholder="Email" required />
```

The same pattern was applied to the username, password, login email, and login password fields. The CSS was also split so the large panel switch labels use `.toggle-label`, while normal input labels use `.field-label`. That avoided turning every form label into a page heading. The final result keeps the field name visible after typing and gives assistive technology a clear label-to-control relationship.

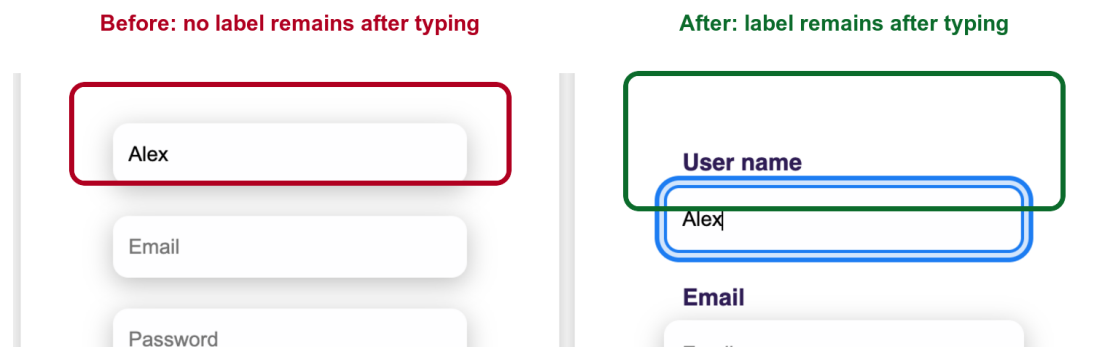


Figure 1: Authentication form label evidence. Before the fix, the username placeholder disappears after typing and no field label remains. After the fix, the visible “User name” label stays above the filled input.

3 Auth Forms Submitted to Missing Backend Routes

3.1 Detection

The next issue came from following the same forms through to the backend. In `views/signup.ejs`, the sign-up form submitted `POST /signup` and the login form submitted `POST /login`. In the earlier `src/app.js`, Express only defined `GET /`, `GET /signup`, and `GET /dashboard`. A direct curl check confirmed the bug: a valid `POST /signup` returned `HTTP/1.1 404 Not Found`. In practice, a user could fill the form correctly and still be sent to a missing route.

3.2 Literature

Express routing is matched by both HTTP method and path, so `GET /signup` does not handle `POST /signup` [3]. OWASP’s Input Validation Cheat Sheet was used because direct POST requests can bypass browser validation, so required fields and email format still need server-side checks [4]. MDN’s 303 See Other documentation was used for the successful response: after a POST, the browser should be redirected to a GET page such as `/dashboard` [5].

3.3 Implementation

The earlier backend rendered the sign-up page but did not process its submission:
Before fix: sign-up page existed, but no POST handler existed.

```
app.get("/signup", (req, res) => {
  res.status(200).render("signup.ejs");
});
```

The fix added method-specific POST handlers for the existing form actions:
After fix: server-side validation and Post/Redirect/Get.

```
app.post("/signup", (req, res) => {
  const missingFields = collectMissingFields(req.body, [
    "SignUpUsername", "SignUpEmail", "SignUpPassword",
  ]);

  if (missingFields.length > 0 || !isValidEmail(req.body.SignUpEmail)) {
    return res.status(400).json({
      success: false,
      errors: {
        missingFields,
        email: isValidEmail(req.body.SignUpEmail) ? undefined
          : "Enter a valid email address.",
      },
    });
  }

  return res.redirect(303, "/dashboard");
});
```

The same approach was added for POST /login. The app now rejects invalid email input with 400 Bad Request and redirects valid submissions with 303 See Other. To make this testable, src/app.js exports the Express app and only starts listen() when run directly, so node --test can run the route checks on a temporary port.

Backend route evidence

Before fix:

POST /signup -> HTTP/1.1 404 Not Found

After fix:

POST /signup -> HTTP/1.1 303 See Other
Location: /dashboard

POST /login -> HTTP/1.1 303 See Other
Location: /dashboard

Invalid email -> HTTP/1.1 400 Bad Request
{ "success": false, "errors": { "email": "Enter a valid email address." } }

Figure 2: Backend route evidence. Before the fix, a valid POST /signup returned 404. After the fix, valid sign-up and login submissions return 303 See Other to /dashboard, while invalid email input is rejected with 400 Bad Request.

4 Deficiency 3 Placeholder

TODO: Insert the third completed deficiency. Include detection method, literature source, implementation rationale, and real before-vs-after code.

5 Deficiency 4 Placeholder

TODO: Insert the fourth completed deficiency. Include detection method, literature source, implementation rationale, and real before-vs-after code.

6 Baseline Standards Evidence

TODO: Add concise evidence and screenshots for the required baseline standards: 7-day Vercel/Render uptime, Codecov/Istanbul coverage of at least 80%, Lighthouse Accessibility score of at least 90, a two-language i18n toggle, a cookie banner, and a privacy policy page. Each screenshot must have a figure number and descriptive caption.

7 Individual Contribution Forms

TODO: Insert or reference the four individual contribution forms. These are excluded from the word count. Add direct links to authored pull requests where required by the marking scheme.

8 References

- [1] W3C Web Accessibility Initiative, “Labeling Controls,” Forms Tutorial. [Online]. Available: <https://www.w3.org/WAI/tutorials/forms/labels/>. Accessed: May 8, 2026.
- [2] WebAIM, “Creating Accessible Forms – Advanced Form Labeling.” [Online]. Available: <https://webaim.org/techniques/forms/advanced>. Accessed: May 8, 2026.
- [3] Express, “Routing.” [Online]. Available: <https://expressjs.com/en/guide/routing.html>. Accessed: May 8, 2026.
- [4] OWASP Cheat Sheet Series, “Input Validation Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html. Accessed: May 8, 2026.
- [5] MDN Web Docs, “303 See Other.” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/303>. Accessed: May 8, 2026.